

Chapter 06: Whiteboard & Google Doc Coding Discipline

Coding Without a Safety Net Modern IDEs (VS Code, PyCharm) make developers comfortable. Red squiggly lines catch typos, tab-autocomplete completes variable names, and pressing "Run" lets you test code instantly.

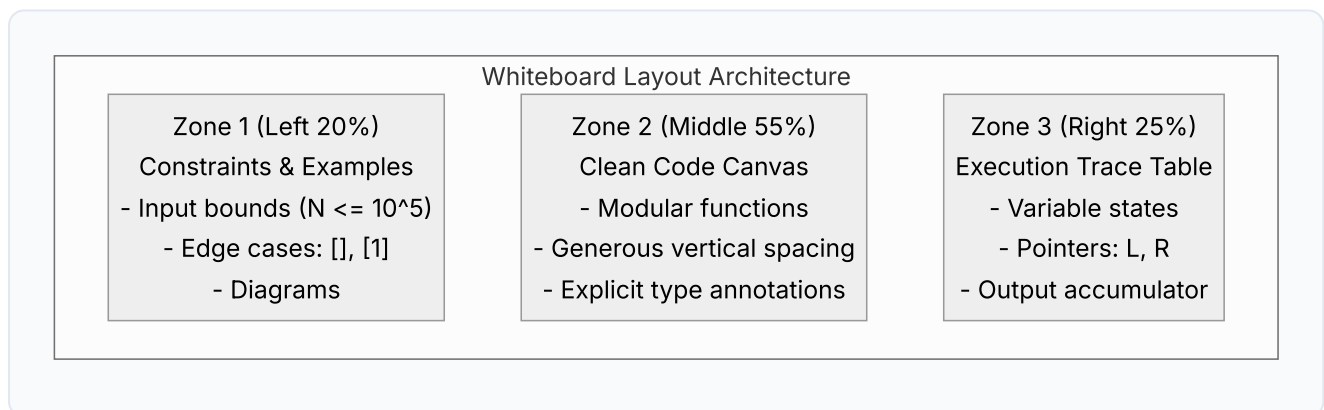
In Google, Meta, and top-tier hedge fund interviews, this safety net vanishes. You are handed a dry-erase marker at a whiteboard, or forced to type in a shared Google Doc without syntax highlighting, autocomplete, or an execution compiler.

This chapter teaches the discipline of **mental compilation**, whiteboard space management, and manual debugging routines.

1. The Whiteboard Space Partitioning Law

When coding on a physical whiteboard or a plain text editor, poor organization leads to running out of room, messy arrows, and crossed-out scribbles.

Divide your canvas into 3 strict zones:



The Rule of Modular Helper Functions

Never write a 60-line monolithic function on a whiteboard. If your solution requires a sub-routine (e.g., verifying if a string is a palindrome, or reversing a linked list):

1. **Stub it out first:** Write `if not self.is_palindrome(s[left:right]):` and assume it works.
2. Implement the core high-level algorithm.
3. If time permits, implement the helper function at the bottom.

4. *Why this works*: It keeps your main logic clean and proves architectural thinking to the interviewer!
-

2. The 10 Most Common Bugs IDEs Catch That You Must Catch Manually

In a Google Doc, the compiler will not warn you about these 10 classic traps. Memorize them:

1. The 2D Matrix Shallow Copy Trap

2. Off-by-One in Binary Search

3. Modifying a Collection While Iterating

4. Unhandled `None` Pointer Access in Linked Lists / Trees

5. Python Integer Division Truncation

6. Missing Return Statement in Recursive Branch

7. Mutable Default Arguments

8. Shadowing Built-In Names

Avoid naming variables `list`, `dict`, `min`, `max`, `sum`, `id`, `type`, or `str`. It overwrites the built-in function, causing silent downstream errors when you call `max(a, b)`.

9. String Concatenation Inside Loops ($O(N^2)$ Trap)



10. Forgetting to Update Pointers in `while` Loops



3. The Mental Dry-Run Execution Protocol

Before you tell the interviewer you have finished, execute this 4-step mental dry-run:



Mastering this discipline separates engineers who rely on trial-and-error from masters who write correct, production-grade code on the first attempt.

Further Reading

- [Python style guide \(PEP 8\)](#)
- [Big-O cheat sheet](#)
- [Python data structures](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. Why write helper functions with clear names on a whiteboard?

They keep the main logic readable and reduce bugs.

▼ 2. What do you state about complexity?

Time and space in terms of input size, including hidden costs like sorting or slicing.

▼ 3. How do you handle an unfamiliar language feature?

Use a simpler construct you are certain about and say so.

▼ 4. What is a good habit before declaring done?

Test one normal case and one edge case aloud.